

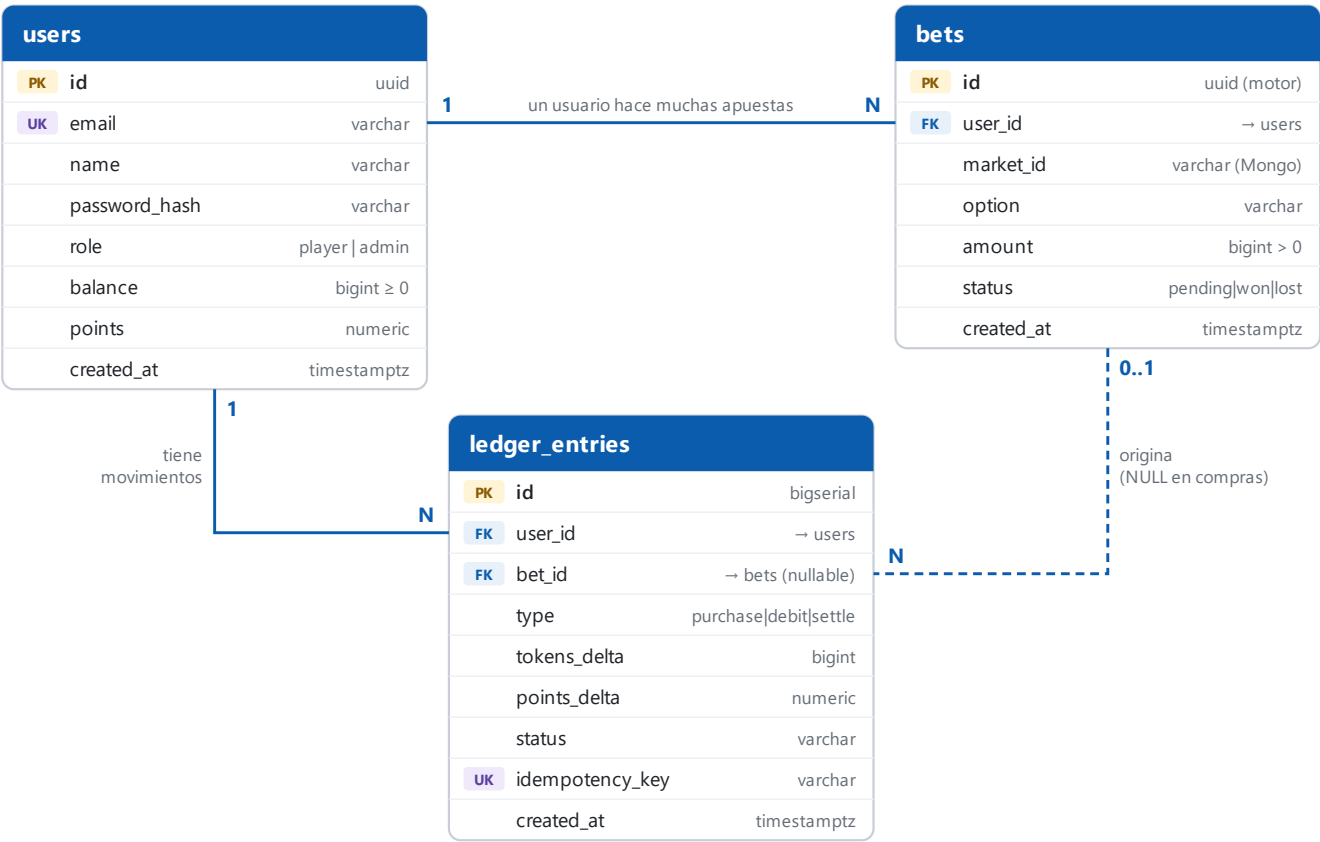
Modelo de datos — Pagos y usuarios

ArquiSoft · Prototipo 1 · PostgreSQL

Estado: propuesta. Usa el modelo de puntos del documento (RF-10): acertar suma 10 + monto × 0,5 puntos y los tokens apostados no se devuelven (pregunta abierta del equipo; si cambia, la liquidación solo agrega `tokens_delta` positivo).

La base de datos de Pagos es el “banco” del sistema: sabe **quién es cada usuario, cuánto tiene y qué movimientos ha hecho**. No sabe nada de carreras, pilotos ni reglas del juego; eso vive en el motor (MongoDB). Para el Prototipo 1 bastan **3 tablas**.

Diagrama



PK llave primaria **FK** llave foránea **UK** valor único · línea punteada = relación opcional

Por qué existe cada tabla

Tabla	Requisito que la exige
users	RF-01, RF-02 y RF-13: registro, inicio de sesión y administrador. También guarda saldo y puntaje.
bets	RF-08 y RNF-01: descontar el saldo y registrar la apuesta deben ser una sola transacción en PostgreSQL.
ledger_entries	RNF-10: cada movimiento de saldo y de puntos queda registrado con usuario, tipo, monto, fecha y estado.

users — quién es y cuánto tiene

Atributo	Tipo	Por qué
id	UUID • PK	Identifica al usuario. Va dentro del JWT y es el <code>user_id</code> que el motor envía a Pagos. UUID para no exponer cuántos usuarios hay ni permitir adivinar ids.
email	VARCHAR(255) • UNIQUE	Con él se inicia sesión (RF-02). Ser único implementa “Este correo ya está registrado” (HU-01).
name	VARCHAR(100)	Lo pide el registro (RF-01) y se muestra en la interfaz.
password_hash	VARCHAR(255)	Solo el hash bcrypt/Argon2, nunca la contraseña en texto plano (RNF-06).
role	VARCHAR(20) • CHECK	<code>player</code> o <code>admin</code> . Va en el JWT: solo el admin abre mercados y solo los jugadores compran y apuestan (RF-13). El registro público siempre crea <code>player</code> .
balance	BIGINT • CHECK ≥ 0	Tokens disponibles. Entero porque nunca se usa <code>float</code> para dinero. El CHECK es la última defensa para que el saldo nunca quede negativo (RNF-01).
points	NUMERIC(12,1)	Puntaje acumulado (RF-10, RF-11). Un decimal basta porque 10 + monto × 0,5 siempre termina en ,0 o ,5.
created_at	TIMESTAMP TZ	Fecha de registro; útil para auditoría y depuración.

bets — el dinero comprometido en cada apuesta

El motor decide sobre la apuesta (mercado abierto o cerrado, quién gana, cuántos puntos) y Pagos guarda el dinero que se comprometió.

Atributo	Tipo	Por qué
id	UUID • PK	Lo genera el motor , así los dos servicios usan el mismo identificador. Como es llave primaria, si el motor reintenta el débito, la apuesta no se duplica.
user_id	UUID • FK → users	Quién apostó: a quién se le descuenta y a quién se le suman puntos.
market_id	VARCHAR(64)	En qué mercado se apostó. Sin llave foránea porque el mercado vive en MongoDB: cada servicio es dueño de su base de datos. Al liquidar, el motor le pide a Pagos (por REST) las apuestas de un mercado; por eso lleva índice.
option	VARCHAR(100)	Qué opción eligió el jugador (p. ej. NOR). Pagos se la devuelve al motor al liquidar; sin esto no se puede saber quién acertó.
amount	BIGINT • CHECK > 0	Tokens apostados. Se descuentan del saldo y alimentan la fórmula de puntos. RF-08 exige que sea mayor que 0.
status	VARCHAR(20) • CHECK	<code>pending</code> → <code>won</code> / <code>lost</code> . Muestra que la apuesta se resolvió y evita liquidarla dos veces: la liquidación solo actualiza apuestas en <code>pending</code> (RNF-02).
created_at	TIMESTAMP TZ	Cuándo se registró la apuesta.

ledger_entries — el historial de todos los movimientos

Es como el extracto bancario: **solo se insertan filas, nunca se editan ni se borran**. Si el sistema está bien, para cada usuario se cumple:

- `users.balance` = suma de `tokens_delta` de sus movimientos
- `users.points` = suma de `points_delta` de sus movimientos

Esa es la verificación de HU-09.

Atributo	Tipo	Por qué
id	BIGSERIAL · PK	Identificador consecutivo; además muestra el orden en que ocurrieron los movimientos.
user_id	UUID · FK → users	De quién es el movimiento (RNF-10: “usuario”).
bet_id	UUID · FK → bets, nullable	Qué apuesta originó el movimiento. <code>NULL</code> en las compras de tokens, que no vienen de ninguna apuesta.
type	VARCHAR(20) · CHECK	<code>purchase</code> (compra), <code>bet_debit</code> (cobro de la apuesta) o <code>bet_settlement</code> (liquidación). Es el “tipo” que pide RNF-10.
tokens_delta	BIGINT	Cambio en tokens: +100 en una compra, -40 en una apuesta, 0 en una liquidación.
points_delta	NUMERIC(12,1)	Cambio en puntos: +30 al acertar, 0 en lo demás (también en una apuesta perdida, que igual deja su fila de liquidación). Va aparte de los tokens porque son unidades distintas y no se deben sumar entre sí.
status	VARCHAR(20)	Lo pide RNF-10. En el Prototipo 1 siempre es <code>completed</code> , porque solo se escribe dentro de transacciones exitosas.
idempotency_key	VARCHAR(255) · UNIQUE	Evita aplicar dos veces la misma operación (RNF-02): un doble clic en “Comprar” o un reintento del motor chocan con la restricción. Formato: <code>purchase:<clave del front></code> , <code>bet_debit:<bet_id></code> , <code>settlement:<bet_id></code> .
created_at	TIMESTAMP TZ	Fecha del movimiento (RNF-10: “fecha”).

Ejemplo: el recorrido de Ana

Ana compra 100 tokens, apuesta 40 a NOR y acierta ($10 + 40 \times 0,5 = 30$ puntos).

Paso	Cambios en users	bets	Fila en ledger_entries
Compra	balance 0 → 100	—	<code>purchase</code> , tokens +100
Apuesta	balance 100 → 60	nueva, pending	<code>bet_debit</code> , tokens -40
Liquidación	points 0 → 30	pending → won	<code>bet_settlement</code> , puntos +30

Si Ana hubiera **fallado**, la liquidación deja la apuesta en `lost` y escribe igual una fila `bet_settlement` con 0 tokens y 0 puntos: HU-07 pide que *cada resultado* quede en el ledger.

Cada paso es **una sola transacción**: o se aplican todos sus cambios o ninguno.

Cómo se protege la concurrencia (8 personas a la vez)

El débito de una apuesta se hace con un único `UPDATE` atómico:

```
UPDATE users SET balance = balance - :monto
WHERE id = :user_id AND balance >= :monto;
```

Si no afecta ninguna fila, la respuesta es “Saldo insuficiente”. Si llegan dos apuestas al mismo tiempo, PostgreSQL bloquea la fila y las procesa una tras otra, así que nunca se gasta el mismo saldo dos veces (RNF-01). El `CHECK (balance >= 0)` queda como red de seguridad.

Simplificaciones del Prototipo 1

Son decisiones a propósito y se pueden revertir en entregas posteriores sin rediseñar:

- **El saldo vive en users** y no en una tabla `wallets` separada. Con 8 usuarios no hace falta separar identidad y dinero.
- **No hay tabla de compras.** Con una tasa fija y un pago simulado, el valor en COP siempre es $\text{tokens} \times \text{COP_PER_TOKEN}$. Cuando haya pasarela real o retiros, se agrega `token_purchases`.
- **No hay tabla de sesiones.** El JWT no la necesita; cerrar sesión es borrar la cookie.
- **La fórmula de puntos no vive en Pagos.** El motor la calcula y Pagos solo aplica los créditos que recibe. Si el equipo cambia la regla de liquidación (por ejemplo, a cuotas tipo casino), Pagos casi no cambia.